

Object-oriented programming

Object-oriented programming involves thinking of problems as interactions between entities.

Object-oriented programming involves thinking of problems as interactions between entities.

A **class** is a set of **objects** (or **instances**) with similar features. A typical kind of class is a **User** or a **Product**, and objects might be **Ashley** or **BananaBunch**. Objects have **properties** (variables that belong to it) and **methods** (functions that belong to it), like **firstName**, **lastName**, or **numberOfBananas**.

Object-oriented programming

Object-oriented programming involves thinking of problems as interactions between entities.

A **class** is a set of **objects** (or **instances**) with similar features. A typical kind of class is a **User** or a **Product**, and objects might be **Ashley** or **BananaBunch**. Objects have **properties** (variables that belong to it) and **methods** (functions that belong to it), like **firstName**, **lastName**, or **numberOfBananas**.

Classes can exist in a hierarchy, where they inherit properties or methods from superclasses.

Object-oriented programming

Object-oriented programming involves thinking of problems as interactions between entities.

A **class** is a set of **objects** (or **instances**) with similar features. A typical kind of class is a **User** or a **Product**, and objects might be **Ashley** or **BananaBunch**. Objects have **properties** (variables that belong to it) and **methods** (functions that belong to it), like **firstName**, **lastName**, or **numberOfBananas**.

Classes can exist in a hierarchy, where they inherit properties or methods from superclasses.

You can think of a class as your own custom type of variable, with data fields (properties) and dedicated functions (methods).

	Python
Constructors	<code>def __init__(self, input):</code>
Display	<code>def __repr__(self):</code>
Getters	<code>@property / def myProperty(self):</code>
Setters	<code>@myProperty.setter / def myProperty(self, value):</code>
Ordinary methods	<code>def someMethod(self, input):</code>
Static methods	<code>@staticmethod / def someMethod(input):</code>

Python	
Public attributes	<code>self.myProperty</code>
Private properties	<code>self.__myPrivateProperty</code>
Constants	Class attribute

Class methods and properties are **scoped to the class**; they can be **public** (visible externally), **private** (meant only for other methods in the class), or **protected** (also meant for child class methods). They can be given a number of attributes.


```
class LabResult:
    def __init__(self, cv):
        self.current_value = cv
        self.assign_status()
    def assign_status(self):
        if self.current_value < 10:
            self.status = "Too low"
        else:
            self.status = "Too high"
    @staticmethod
    def load_object(s):
        return LabResult(s)
```

Python is very widely used and many people contribute code to it. Function name clashes are somewhat inevitable. This is where modules come in handy.

Python is very widely used and many people contribute code to it. Function name clashes are somewhat inevitable. This is where modules come in handy.

A module is just a folder.

Python is very widely used and many people contribute code to it. Function name clashes are somewhat inevitable. This is where modules come in handy.

A module is just a folder.

You can import modules the regular way. In general, do not `from myModule import ...`; import the whole module and use `myModule.myFunction()`. That is less prone to errors.

Classes are most useful when you have multiple functions that operate on a similar kind of data or type of variable.

Classes are most useful when you have multiple functions that operate on a similar kind of data or type of variable.

Modules are most useful when you have many functions that functionally belong together but are not necessarily associated with a particular kind of input.

- Input validation is the process of checking if input data is valid before using it.

- Input validation is the process of checking if input data is valid before using it.
- It helps ensure that your program runs correctly and securely.

- Input validation is the process of checking if input data is valid before using it.
- It helps ensure that your program runs correctly and securely.
- Important if you end up with a large code base and no longer expect the user (that's you) to be looking at every part of the code every time they use it.

- Input validation is the process of checking if input data is valid before using it.
- It helps ensure that your program runs correctly and securely.
- Important if you end up with a large code base and no longer expect the user (that's you) to be looking at every part of the code every time they use it.
- Common types of input validation include range checking, type checking, format checking, and existence checking.

- Helps prevent unexpected behavior.

Benefits of Input Validation

- Helps prevent unexpected behavior.
- Improves code readability and maintainability.

Benefits of Input Validation

- Helps prevent unexpected behavior.
- Improves code readability and maintainability.
- Reduces security risks.

Benefits of Input Validation

- Helps prevent unexpected behavior.
- Improves code readability and maintainability.
- Reduces security risks.
- Provides better user experience by giving informative error messages.

- Range checking: Ensures input falls within acceptable range of values.

Common Types of Input Validation

- Range checking: Ensures input falls within acceptable range of values.
- Type checking: Ensures input is of the expected type.

Common Types of Input Validation

- Range checking: Ensures input falls within acceptable range of values.
- Type checking: Ensures input is of the expected type.
- Format checking: Ensures input conforms to a specific format (e.g., phone numbers or email addresses).

Common Types of Input Validation

- Range checking: Ensures input falls within acceptable range of values.
- Type checking: Ensures input is of the expected type.
- Format checking: Ensures input conforms to a specific format (e.g., phone numbers or email addresses).
- Existence checking: Ensures input is not empty or null.

Basic Input Validation Example

```
class BankAccount:
    def __init__(self, owner, balance):
        if balance < 0:
            raise ValueError("Balance cannot be negative.")
        self.owner = owner
        self.balance = balance
    # ...
```

- Provide informative error messages to the user.

- Provide informative error messages to the user.
- Log error messages for developers to review.

Handling Invalid Inputs Gracefully

- Provide informative error messages to the user.
- Log error messages for developers to review.
- Raise exceptions with descriptive error messages.

Handling Invalid Inputs Gracefully

- Provide informative error messages to the user.
- Log error messages for developers to review.
- Raise exceptions with descriptive error messages.
- Ensure that the program does not continue with invalid input.

- Input validation can be incorporated into methods and class constructors.

- Input validation can be incorporated into methods and class constructors.
- It's a good practice to perform input validation as close to the source of input as possible.

- Input validation can be incorporated into methods and class constructors.
- It's a good practice to perform input validation as close to the source of input as possible.
- Encapsulation allows you to define rules for valid state and prevent invalid input from corrupting the state of the object.

A corrupted state is a situation where an object's data is in an inconsistent or invalid state, often caused by an error in the program or a violation of the object's invariants.

A corrupted state is a situation where an object's data is in an inconsistent or invalid state, often caused by an error in the program or a violation of the object's invariants.

- An object is in a corrupted state when its internal data is inconsistent with the object's intended behavior

A corrupted state is a situation where an object's data is in an inconsistent or invalid state, often caused by an error in the program or a violation of the object's invariants.

- An object is in a corrupted state when its internal data is inconsistent with the object's intended behavior
- A corrupted state can lead to bugs and errors in the program

A corrupted state is a situation where an object's data is in an inconsistent or invalid state, often caused by an error in the program or a violation of the object's invariants.

- An object is in a corrupted state when its internal data is inconsistent with the object's intended behavior
- A corrupted state can lead to bugs and errors in the program
- Preventing corrupted states is an important part of writing correct, reliable code

Example: Workflow that leads to a Corrupted State

Consider this object, with a balance attribute and a deposit and withdraw method.

Example: Workflow that leads to a Corrupted State

Consider this object, with a balance attribute and a deposit and withdraw method.

```
class BankAccount:
    def __init__(self, balance):
        if balance < 0:
            raise ValueError("Balance must be positive.")
        self.balance = balance
    def deposit(self, amount):
        self.balance += amount
    def withdraw(self, amount):
        if amount > self.balance:
            raise ValueError("Overdrawing not allowed!")
        self.balance -= amount
```


Example: Workflow that leads to a Corrupted State

Suppose we have the following workflow:

Example: Workflow that leads to a Corrupted State

Suppose we have the following workflow:

1. Create a new bank account with initial balance of 100 dollars

Example: Workflow that leads to a Corrupted State

Suppose we have the following workflow:

1. Create a new bank account with initial balance of 100 dollars
2. Make a deposit of -50 dollars

Example: Workflow that leads to a Corrupted State

Suppose we have the following workflow:

1. Create a new bank account with initial balance of 100 dollars
2. Make a deposit of -50 dollars
3. That's arguably not a deposit

Example: Workflow that leads to a Corrupted State

Suppose we have the following workflow:

1. Create a new bank account with initial balance of 100 dollars
2. Make a deposit of -50 dollars
3. That's arguably not a deposit
4. This workflow violates the invariants of the bank account object and results in a corrupted state, where the balance is negative.

Example: External Function Bypassing Logger Method

```
class BankAccount:
    def __init__(self, balance):
        self.balance = balance
        self.history = []
    def deposit(self, amount):
        self.balance += amount
        self.history.append(('deposit', amount))
    def withdraw(self, amount):
        self.balance -= amount
        self.history.append(('withdraw', amount))
    def logger(self):
        for transaction in self.history:
            print(transaction)
```

Example: External Function Bypassing Logger Method

Now suppose an **external** function edits the balance attribute of the BankAccount object directly, bypassing the deposit and withdraw methods and the logger method:

Example: External Function Bypassing Logger Method

Now suppose an **external** function edits the balance attribute of the BankAccount object directly, bypassing the deposit and withdraw methods and the logger method:

```
def yoink(account):  
    account.balance -= 100
```


Example: External Function Bypassing Logger Method

Now suppose an **external** function edits the balance attribute of the BankAccount object directly, bypassing the deposit and withdraw methods and the logger method:

```
def yoink(account):  
    account.balance -= 100
```

This bypasses the logger method and leads to a corrupted state where the transaction history no longer reflects the actual transactions made on the account.

Public methods and attributes are accessible from outside the class.

Private vs Public in OOP

Public methods and attributes are accessible from outside the class.

Private methods and attributes are only accessible within the class.

Private vs Public in OOP

Public methods and attributes are accessible from outside the class.

Private methods and attributes are only accessible within the class.

```
class MyClass:
    def __init__(self):
        self.public_attr = 0
        self.__private_attr = 1
    def public_method(self):
        print("This is a public method")
        self.__private_method()
    def __private_method(self):
        print("This is a private method")
```

- By convention, two leading underscores indicate a “private” method or attribute

- By convention, two leading underscores indicate a “private” method or attribute
- Private methods and attributes are intended for internal use only, and should not be accessed from outside the class

- By convention, two leading underscores indicate a “private” method or attribute
- Private methods and attributes are intended for internal use only, and should not be accessed from outside the class
- Public methods and attributes can be freely accessed from outside the class

- By convention, two leading underscores indicate a “private” method or attribute
- Private methods and attributes are intended for internal use only, and should not be accessed from outside the class
- Public methods and attributes can be freely accessed from outside the class
- In Python, there are no **truly** private methods or attributes

Private Attributes in the BankAccount Class

```
class BankAccount:
    def __init__(self, initial_balance):
        if initial_balance < 0:
            raise ValueError("Initial balance cannot be <0")
        self.__balance = initial_balance

    def get_balance(self):
        return self.__balance
```

- The `__balance` attribute in `BankAccount` is now private

Private Attributes in the BankAccount Class

- The `__balance` attribute in `BankAccount` is now private
- It is accessed using the `get_balance()` method, which is public

- The `__balance` attribute in `BankAccount` is now private
- It is accessed using the `get_balance()` method, which is public
- External code shouldn't modify the `__balance` attribute directly

Private Attributes in the BankAccount Class

- The `__balance` attribute in `BankAccount` is now private
- It is accessed using the `get_balance()` method, which is public
- External code shouldn't modify the `__balance` attribute directly
- This ensures that the balance is always valid and prevents external code from corrupting the object's state

```
class BankAccount:
    def __init__(self, initial_balance):
        if initial_balance < 0:
            raise ValueError("Initial balance cannot be <0")
        self.__balance = initial_balance

    def get_balance(self):
        return self.__balance
```

Public Methods in the BankAccount Class

```
class BankAccount:
    def deposit(self, amount):
        if amount < 0:
            raise ValueError("Deposit amount cannot be <0")
        self.__balance += amount

    def withdraw(self, amount):
        if amount < 0:
            raise ValueError("Withdrawal amount cannot be <0")
        if self.__balance < amount:
            raise ValueError("Insufficient funds")
        self.__balance -= amount
```

- The `deposit()` and `withdraw()` methods in `BankAccount` are public

- The `deposit()` and `withdraw()` methods in `BankAccount` are public
- They allow external code to modify the object's state in a controlled manner

Public Methods in the BankAccount Class

- The `deposit()` and `withdraw()` methods in `BankAccount` are public
- They allow external code to modify the object's state in a controlled manner
- They perform validation to ensure that the object remains in a valid state

Public Methods in the BankAccount Class

- The `deposit()` and `withdraw()` methods in `BankAccount` are public
- They allow external code to modify the object's state in a controlled manner
- They perform validation to ensure that the object remains in a valid state
- External code cannot access the `balance` attribute directly, so it cannot corrupt the object's state

Operator Overloading in Object-Oriented Programming

Operator overloading is a technique in object-oriented programming that allows us to define the behavior of operators (+, -, *, /, etc.) for user-defined objects.

Operator Overloading in Object-Oriented Programming

Operator overloading is a technique in object-oriented programming that allows us to define the behavior of operators (+, -, *, /, etc.) for user-defined objects.

In Python, operator overloading is achieved by defining special methods (with double underscores) in a class that correspond to the operator being used.

Operator Overloading in Object-Oriented Programming

Operator overloading is a technique in object-oriented programming that allows us to define the behavior of operators (+, -, *, /, etc.) for user-defined objects.

In Python, operator overloading is achieved by defining special methods (with double underscores) in a class that correspond to the operator being used.

For example, the addition operator (+) can be overloaded by defining the `__add__` method in a class.

Suppose we have a class called **Vector** that represents a mathematical vector in 2D space. We can overload the addition operator (+) to allow adding two vectors together.

Operator Overloading in Object-Oriented Programming

Suppose we have a class called **Vector** that represents a mathematical vector in 2D space. We can overload the addition operator (+) to allow adding two vectors together.

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)
```


Let's create two **Vector** objects and add them together using the overloaded addition operator (+):

Using Overloaded Operators

Let's create two **Vector** objects and add them together using the overloaded addition operator (+):

```
v1 = Vector(1, 2)
```

```
v2 = Vector(3, 4)
```

```
v3 = v1 + v2
```

```
print(v3.x, v3.y)
```

Using Overloaded Operators

Let's create two **Vector** objects and add them together using the overloaded addition operator (+):

```
v1 = Vector(1, 2)
v2 = Vector(3, 4)
v3 = v1 + v2
print(v3.x, v3.y)
```

Output: 4 6

Using Overloaded Operators

Let's create two **Vector** objects and add them together using the overloaded addition operator (+):

```
v1 = Vector(1, 2)
v2 = Vector(3, 4)
v3 = v1 + v2
print(v3.x, v3.y)
```

Output: 4 6

Here, we create two **Vector** objects, **v1** and **v2**, and add them together using the overloaded addition operator **+**. The resulting **Vector** object, **v3**, has components (4, 6).

Operator Overloading in Object-Oriented Programming

We can also overload other operators, such as the multiplication operator (*), to perform scalar multiplication of a vector.

Operator Overloading in Object-Oriented Programming

We can also overload other operators, such as the multiplication operator (*), to perform scalar multiplication of a vector.

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)
    def __mul__(self, scalar):
        return Vector(self.x * scalar, self.y * scalar)
```

Operator Overloading in Object-Oriented Programming

We can also overload other operators, such as the multiplication operator (*), to perform scalar multiplication of a vector.

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)
    def __mul__(self, scalar):
        return Vector(self.x * scalar, self.y * scalar)
```

By overloading operators, we can make our code more intuitive and expressive, and allow our user-defined objects to behave more like built-in types.

List of Overloaded Operators in Python

Operator	Method Name	Description
+	<code>__add__</code>	Addition
-	<code>__sub__</code>	Subtraction
*	<code>__mul__</code>	Multiplication
/	<code>__truediv__</code>	Division
//	<code>__floordiv__</code>	Floor Division
%	<code>__mod__</code>	Modulo

List of Overloaded Operators in Python

Operator	Method Name	Description
<code>**</code>	<code>__pow__</code>	Exponentiation
<code><</code>	<code>__lt__</code>	Less Than
<code><=</code>	<code>__le__</code>	Less Than or Equal To
<code>==</code>	<code>__eq__</code>	Equal To
<code>!=</code>	<code>__ne__</code>	Not Equal To
<code>></code>	<code>__gt__</code>	Greater Than
<code>>=</code>	<code>__ge__</code>	Greater Than or Equal To

Example object for normal distribution functions

```
import math

class NormalDistribution:
    def __init__(self, mu=0.0, sigma=1.0):
        if sigma <= 0:
            raise ValueError("sigma must be positive")
        self.mu = float(mu)
        self.sigma = float(sigma)

    def z(self, x):
        return (x - self.mu) / self.sigma
```

A worked object for normal distribution functions

```
...  
def pdf(self, x):  
    z = self.z(x)  
    return (  
        math.exp(-0.5 * z * z)  
        / (self.sigma * math.sqrt(2.0 * math.pi))  
    )  
  
def cdf(self, x):  
    z = self.z(x)  
    return 0.5 * (1.0 + math.erf(z / math.sqrt(2.0)))
```

A worked object for normal distribution functions

```
nd = NormalDistribution(mu=0.0, sigma=1.0)

print(nd.pdf(0.0))    # 0.3989422804014327
print(nd.cdf(0.0))    # 0.5
print(nd.cdf(1.96))   # about 0.975
```

A worked object for normal distribution functions

```
nd = NormalDistribution(mu=0.0, sigma=1.0)

print(nd.pdf(0.0))    # 0.3989422804014327
print(nd.cdf(0.0))    # 0.5
print(nd.cdf(1.96))   # about 0.975
```

This object keeps parameters and distribution functions together in one reusable interface.